PROJECT DOCUMENT

Hospital Management System (HMS)

Project: Hospital Management System – Full Stack Implementation

Document Type: Project Implementation Record

Date: July 2026

1. Project Overview

The Hospital Management System (HMS) is a full-stack digital platform developed to streamline and automate core hospital operations. It consists of three integrated components: a Node.js/Express backend, an Angular web frontend, and a React Native mobile application. Together, these components provide a comprehensive, role-based system for managing patients, employees, appointments, medical records, and system access.

1.1 Purpose

The HMS platform serves as the central system for hospital operations. The backend functions as the API server, the web frontend provides the administrative and staff interface, and the mobile application delivers a patient-facing experience on smartphones. All three layers are tightly integrated through a shared API design and role-based authorization model.

1.2 System Components

- **Backend (Node.js / Express):** Provides RESTful APIs for all business operations including authentication, patient management, appointment handling, and medical records.
- **Frontend (Angular 21):** Web-based management interface for administrators, receptionists, and doctors to manage all HMS operations.
- **Mobile Application (React Native / Expo):** Patient-facing mobile app for appointment booking, medical history viewing, and profile management.

1.3 Business Alignment

This implementation directly supports the business objectives defined in the HMS Business Requirements Document (BRD), including:

- Role-based CRUD operations across all modules
- Approval workflows for employee self-signup and patient appointment requests
- Secure authentication using JWT tokens
- Role-based page and API access control
- Comprehensive management of Employee, Patient, Appointment, and Health Record modules

2. Backend Architecture

The backend is a Node.js and Express.js application that acts as the central API server for the HMS platform. It manages authentication, authorization, data access, and business logic for all system operations.

2.1 Technology Stack

Technology	Purpose
Node.js	Server-side JavaScript runtime
Express.js	Web framework for routing and middleware
MongoDB	NoSQL database for all data storage
Mongoose	ODM library for MongoDB schema management
JSON Web Tokens (JWT)	Authentication and authorization tokens
express-validator	Request input validation
Brevo	Email notification and verification
Helmet	HTTP security headers middleware
Morgan	HTTP request logging
CORS	Cross-Origin Resource Sharing control

2.2 Architecture Design

The backend follows a layered architecture pattern with clear separation of concerns across five distinct layers:

2.2.1 Entry Layer

The application is initialized in src/app.js and src/server.js. All global middleware including CORS, Helmet, JSON parsing, Morgan logging, and cookie parsing are registered at this level.

2.2.2 Routing Layer

Route modules in src/routes define the available API endpoints. Each file groups endpoints by business domain — authentication, users, admin, appointments, and medical records.

2.2.3 Controller Layer

Controllers in src/controller contain all request handling logic. They receive HTTP requests, apply validation and business checks, interact with Mongoose models, and return JSON responses.

2.2.4 Model Layer

Mongoose models in src/models define the MongoDB schema and data structures. They include validation rules, indexes, and pre-save hooks for auto-generated IDs.

2.2.5 Middleware and Validation Layer

Middleware in src/middleware handles authentication, permission checking, validation, and centralized error handling. Validation rules are defined separately in src/validation and attached to routes.

2.3 Folder Structure

Folder	Purpose
src/controller	Request handlers for each functional area
src/middleware	Reusable middleware for auth, permissions, validation, and error handling
src/models	Mongoose schemas defining all database collections
src/routes	API endpoint definitions grouped by business domain
src/utils	Shared helpers: error factory, async handler, mail utility, token generator
src/validation	express-validator rule sets attached to routes

2.4 API Design

The backend exposes REST APIs mounted on the following base paths:

- /auth — Authentication and token management
- /user — User and patient profile operations
- /admin — Admin dashboard and employee management
- /appointment — Appointment lifecycle management
- /medicalRecord — Medical record CRUD operations
- /ui — Reference data: roles, departments, specializations
- /node — Role-based navigation node metadata

2.4.1 Authentication Endpoints

Method	Route	Description
POST	/auth/signUp	Register a new employee user
POST	/auth/login	Authenticate and issue JWT tokens

Method	Route	Description
POST	/auth/set-password	Set password for first-time users
GET	/auth/verify-email	Verify user email via token
POST	/auth/patientSignUp	Register a patient user
GET	/auth/getPermissions	Retrieve permissions for a role
GET	/auth/refresh-token	Refresh or retrieve access token
POST	/auth/reset-password	Submit new password using a valid reset token
GET	/auth/verify-reset-password	Validate reset token and confirm identity before allowing password reset

2.4.2 User and Patient Endpoints

Method	Route	Description
GET	/user/getUserProfile	Retrieve employee profile details
GET	/user/getPatients	List patients with pagination
POST	/user/deletePatient	Delete a patient and associated user
GET	/user/getPatientProfile	Retrieve patient profile details
GET	/user/getPatientId	Retrieve patient ID by email
GET	/user/getAvailableTimeSlots	Get free time slots for a doctor on a date
POST	/user/updatePatientProfile	Update a patient profile
GET	/user/getPatientsBySearch	Search patients by ID or name
GET	/user/getDoctorsBySearch	Search doctors by ID, name, or specialization
GET	/user/getPatientById	Fetch a patient by patient ID
GET	/user/getDoctorById	Fetch a doctor by employee ID
GET	/user/getSingleUser	Retrieve a single user profile

2.4.3 Admin Endpoints

Method	Route	Description
POST	/admin/deleteUserProfile	Delete an employee account
GET	/admin/getDashBoardData	Fetch dashboard statistics
GET	/admin/getAllUsers	Retrieve employee records with pagination
GET	/admin/getUsers	Retrieve all user records
GET	/admin/getUserEmployee	Retrieve employee-user combined information

Method	Route	Description
POST	/admin/approveUser	Activate a pending user account
POST	/admin/rejectUser	Reject or deactivate a user account
POST	/admin/updateUserProfile	Update employee profile details

2.4.4 Appointment Endpoints

Method	Route	Description
POST	/appointment/createAppointment	Create a new appointment
GET	/appointment/getAllAppointments	List all appointments with pagination
GET	/appointment/getDoctors	List active doctors
GET	/appointment/getAppointmentUiData	Retrieve appointment summary counts
GET	/appointment/deleteAppointment	Delete an appointment
GET	/appointment/getAppointmentsByPatientId	Retrieve appointments for a given patient
GET	/appointment/getDoctorByEmployeeId	Retrieve doctor details by employee ID
POST	/appointment/editAppointment	Edit an existing appointment
POST	/appointment/editAppointmentStatus	Update appointment status
GET	/appointment/getAppointmentByDoctorIdOrPatientId	Get completed appointments by doctor or patient

2.4.5 Medical Record Endpoints

Method	Route	Description
POST	/medicalRecord/createMedicalRecord	Create a medical record
POST	/medicalRecord/updateMedicalRecord	Update an existing medical record
GET	/medicalRecord/getMedicalRecordStats	Fetch medical record statistics
GET	/medicalRecord/getMedicalRecords	Retrieve medical records with pagination
GET	/medicalRecord/getMedicalRecordById	Retrieve a single medical record
POST	/medicalRecord/deleteMedicalRecord	Soft delete a medical record

2.4.6 Metadata and Navigation Endpoints

Method	Route	Description
GET	/node/getNodes	Fetch UI navigation nodes for a role
GET	/ui/getRoles	Retrieve available roles
GET	/ui/getDepartments	Retrieve available departments
GET	/ui/getSpecializations	Retrieve available specializations

2.5 Database Design

The backend uses MongoDB as its database, accessed through Mongoose. The following collections form the core data model:

2.5.1 Data Models

Model	Collection Purpose	Key Fields
User	Login credentials and account state for all users	email, passwordHash, role, status, employeeId, patientId, isVerified, firstLogin
Employee	Staff and doctor profile information	employeeCode, name, department, designation, specialization, availabilitySlots, isDeleted
Patient	Patient profile information	uhid, name, phone, email, gender, dob, bloodGroup, allergies, address, emergencyContact
Appointment	Doctor-patient appointment records	appointmentId, patientId, doctorEmployeeId, date, timeSlot, status
Medical Record	Diagnosis and treatment details	medicalRecordId, doctorId, appointmentId, patientId, complaint, diagnosis, medications, isDeleted
Role	Role definitions and permissions	role_id, role_name, role_permissions
Department	Department reference data	department_id, department_name
Specialization	Doctor specialization reference data	specialization_id, specialization_name
Node	Frontend navigation definitions by role	order, name, path, role, icon
Counter	Auto-increment sequences for IDs	name, seq

2.5.2 Entity Relationships

- User → Employee: a user may be linked to an employee record via employeeId
- User → Patient: a user may be linked to a patient record via patientId
- Appointment → Patient: each appointment belongs to one patient
- Appointment → Doctor: each appointment is assigned to one doctor employee
- Medical Record → Appointment: each record is linked to a specific appointment
- Medical Record → Patient & Doctor: each record also references the patient and treating doctor
- Role → Permissions: a role has a list of permissions that determine API and page access

2.6 Middleware

Middleware	File	Purpose
Authentication	src/middleware/auth.middleware.js	Verifies JWT Bearer token from the Authorization header and attaches decoded user to req.user
Permission	src/middleware/permission.middleware.js	Checks whether the authenticated user's role has the required permission for the route
Validation	src/middleware/validate.middleware.js	Uses express-validator to inspect request data and returns a 422 error if validation fails
Error Handler	src/middleware/errorHandler.middleware.js	Catches thrown errors and returns a consistent JSON error response with status and message

2.7 Request Lifecycle

A typical API request moves through the backend in the following order:

- 1. Client sends an HTTP request to a defined route
- 2. Validation middleware checks request data format and required fields
- 3. Authentication middleware verifies the JWT token if the route is protected
- 4. Permission middleware checks the user's role has access to the route
- 5. Controller function executes the business logic
- 6. Controller reads or writes data through Mongoose models
- 7. Controller returns a structured JSON response
- 8. Error handler middleware catches and formats any thrown errors

3. Frontend Architecture

The frontend is an Angular 21 web application that serves as the management interface for hospital staff. It provides role-based access to all HMS operational modules including dashboard, employee management, patient management, appointments, approvals, and medical records.

3.1 Technology Stack

Technology	Purpose
Angular 21	Core frontend framework
TypeScript	Strongly typed component and service development
Angular Router	Client-side navigation and route protection
Angular HttpClient	HTTP communication with the backend API
Angular Reactive Forms	Form management and validation
Angular Material Autocomplete	Enhanced input components
RxJS	Reactive programming and HTTP stream handling
ngx-toastr	User-facing toast notification messages
Bootstrap Icons	Icon library for UI elements
LocalStorage	Client-side storage for JWT tokens, role, and permissions

3.2 Application Structure

Folder	Purpose
src/app/theme/login	Login page component
src/app/theme/signup	Employee registration page
src/app/theme/home	Main application shell with header, sidebar, and router outlet
src/app/theme/home/dashboard	Dashboard statistics page
src/app/theme/home/appointment	Appointment management page
src/app/theme/home/employee	Employee list and management page
src/app/theme/home/patient	Patient management page
src/app/theme/home/user-profile	Logged-in user profile page
src/app/theme/home/approval	Pending user approval page
src/app/theme/home/medical-record	Medical record management page

Folder	Purpose
src/app/theme/home/modal	Modal and route-based detail/edit components
src/app/services	Angular services for API communication and state management
src/app/models	TypeScript interfaces and data models
src/app/validators	Custom Angular form validators
src/app/directive	Custom Angular directives including permission-based UI control
src/app/environment	Environment configuration and backend base URL

3.3 Pages and Components

3.3.1 Authentication Pages

Page	File	Purpose
Login	src/app/theme/login/login.ts	Authenticates users via email and password, stores JWT and role in localStorage, and redirects to the appropriate page
Sign Up	src/app/theme/signup/signup.ts	Allows new employee users to register. Loads roles, departments, and specializations from the backend
Forgot Password	src/app/theme/forgot-password/forgot-password.ts	Allows users and patients to request a password reset via email. Publicly accessible — no route guard applied. Added in Sprint 5.

3.3.2 Core Application Pages

Page	File	Purpose
Home Shell	src/app/theme/home/home.ts	Main layout container with header, sidebar, and router outlet. Redirects unauthenticated users to login
Dashboard	src/app/theme/home/dashboard/dashboard.ts	Displays system statistics loaded from the admin API
Appointment	src/app/theme/home/appointment/appointment.ts	Appointment creation, update, deletion, and status

Page	File	Purpose
		management with doctor availability filtering
Employee	src/app/theme/home/employee/employee.ts	Employee list with pagination, department filtering, search, and delete/edit navigation
Patient	src/app/theme/home/patient/patient.ts	Patient registration, listing, search, editing, and deletion
User Profile	src/app/theme/home/user-profile/user-profile.ts	Displays the logged-in user's profile loaded from the backend
Approval	src/app/theme/home/approval/approval.ts	Admin review of pending user accounts with approve or reject actions
Medical Record	src/app/theme/home/medical-record/medical-record.ts	Medical record creation, update, draft and completed states, paginated list

3.3.3 Modal and Detail Components

Component	Purpose
modal/password-modal	First-login password setup flow
modal/edit-employee	Edit employee profile form
modal/edit-patient	Edit patient profile form
modal/view-medical-record	View a full medical record in detail
modal/access-denied	Displayed when a user lacks permission for a page
modal/not-found	404 not found page

3.4 Services and API Integration

All backend communication is handled through Angular services using HttpClient. The base API URL is configured in the environment file.

Service	Backend Endpoints Used	Responsibility
auth.service.ts	/auth	Login, signup, password setup, permission lookup, token refresh

Service	Backend Endpoints Used	Responsibility
user.service.ts	/user, /auth	User profile, patient management, search, profile update
appointment.service.ts	/appointment	Create, list, edit, delete, and query appointments
admin.service.ts	/admin	Dashboard statistics, employee data, approval actions, profile updates
medical-record.service.ts	/medicalRecord	Medical record creation, update, retrieval, and deletion

3.4.1 Authentication Interceptor

The auth interceptor (src/app/services/interceptor/auth.interceptor.ts) automatically attaches the JWT access token as an Authorization header to all outgoing API requests. It also handles token refresh when the access token expires.

3.4.2 Route Guard

The route guard (src/app/services/guard/route.guard.ts) checks the logged-in user's permissions before allowing navigation to protected pages. If access is denied, the user is redirected to the access-denied page.

3.5 User Flows

3.5.1 Login and Role-Based Navigation

- User opens the login page and submits email and password
- Angular login component sends credentials to the backend through the auth service
- On success, the app stores the JWT token, email, role, and permissions in localStorage
- The home shell loads the sidebar dynamically based on navigation nodes from the backend
- The route guard checks permissions before allowing access to each protected page
- Users who have forgotten their password can access the /forgot-password route without logging in; the route is publicly accessible and not protected by the route guard

3.5.2 Admin Flow

- Admin logs in and sees dashboard statistics
- Admin can manage employees, patients, and review pending approval requests
- Admin can approve or reject user accounts and update employee profiles

3.5.3 Staff and Doctor Flow

- Staff logs in and accesses operational pages based on role permissions
- Receptionists can create and manage appointments and patients
- Doctors can manage medical records and record diagnosis details for completed appointments

4. Mobile Application Architecture

The React Native mobile application is the patient-facing client for the HMS platform. It allows patients to log in, browse doctors, book and manage appointments, view medical records, and update their profile from a smartphone.

4.1 Technology Stack

Technology	Purpose
React Native	Cross-platform mobile framework
Expo	Development toolchain and build platform
TypeScript	Strongly typed component and service development
React Navigation (Native Stack)	App-level stack navigation
React Navigation (Bottom Tab)	Main app tab navigation
Axios	HTTP communication with the backend API
Expo Secure Store	Secure JWT token storage
AsyncStorage	Patient email and patient ID persistence across app restarts
Expo Vector Icons	Icon library for mobile UI
React Native Toast Message	User-facing feedback messages
DateTimePicker	Date and time selection in forms
Expo Font	Custom font loading

4.2 Application Structure

Folder / File	Purpose
App.tsx	Application entry point: boots the app, loads fonts, mounts the navigator, and renders the global toast provider
src/navigation/AppNavigator.tsx	Root stack navigator defining all app-level screens
src/navigation/TabNavigator.tsx	Bottom tab navigator for Home, Profile, Appointment, and Medical Records
src/screens	All application screens (login, signup, home, appointment, profile, medical records)
src/components	Reusable UI blocks: input cards, profile cards, doctor cards, form headers, auth buttons
src/services	Backend communication services with Axios
src/hooks	Custom hooks for shared form and business logic

Folder / File	Purpose
src/utils	Validators and shared helper functions
src/types	TypeScript interfaces for users, appointments, medical records, auth, and navigation

4.3 Screens

Screen	File	Purpose
Login	src/screens/login.tsx	Authenticates a patient using email and password and redirects to the main app shell
Sign Up	src/screens/signup.tsx	Registers a new patient account and captures personal and contact details
Home	src/screens/home.tsx	Doctor browsing and search with specialization filters
Appointment	src/screens/appointment.tsx	Book a new appointment by selecting a doctor, date, and available time slot
View Appointment	src/screens/view-appointment.tsx	Lists all appointments for the logged-in patient
Edit Appointment	src/screens/edit-appointment.tsx	Modify or cancel an existing appointment
Profile	src/screens/profile.tsx	View patient personal information and account actions
Edit Profile	src/screens/edit-profile.tsx	Update profile data including name, gender, address, and emergency contact
View Medical Record	src/screens/view-medical-record.tsx	Paginated medical record history with expandable details for complaints, symptoms, diagnosis, and medications

4.4 Navigation Structure

The application uses a nested navigation structure:

4.4.1 Stack Navigation (Root)

The root stack navigator includes: login, signup, tabs (main app shell), viewAppointment, editAppointment, and editProfile.

4.4.2 Bottom Tab Navigation

The tab navigator provides four main sections accessible from the app bottom bar:

- Home — Doctor browsing and appointment initiation
- Profile — Patient profile viewing and editing
- Appointment — Appointment listing and management
- Record — Medical record history

4.5 Backend API Integration

All backend communication is routed through src/services/interceptor.service.ts, which attaches the JWT token to protected request headers and handles API error responses.

4.5.1 Authentication Endpoints

Endpoint	Purpose
auth/login	Authenticate a patient with email and password
auth/patientSignUp	Register a new patient account

4.5.2 User and Profile Endpoints

Endpoint	Purpose
user/getPatientProfile	Fetch the patient's full profile details
user/getPatientId	Retrieve the patient ID from the stored email
user/getAvailableTimeSlots	Fetch available appointment time slots for a doctor on a selected date
user/updatePatientProfile	Submit updated patient profile data

4.5.3 Appointment Endpoints

Endpoint	Purpose
appointment/createAppointment	Create a new appointment for the patient
appointment/getAppointmentsByPatientId	Retrieve all appointments for the logged-in patient
appointment/editAppointment	Modify appointment details (doctor, date, time)
appointment/editAppointmentStatus	Cancel or change appointment status
appointment/getDoctors	Retrieve the list of available doctors

4.5.4 Medical Record and UI Endpoints

Endpoint	Purpose
medicalRecord/getMedicalRecords	Retrieve paginated medical records for the patient
ui/getSpecializations	Retrieve available doctor specializations for filtering

4.6 Storage Strategy

Data	Storage Mechanism	Reason
JWT Access Token	Expo Secure Store	Encrypted storage for sensitive authentication tokens
Patient Email	AsyncStorage	Persistent lightweight storage for session identification
Patient ID	AsyncStorage	Required for appointment and profile API calls across sessions

4.7 Form Validation

The app performs local validation on all forms before sending requests to the backend:

Form	Validated Fields
Login	Email format, password minimum length
Sign Up	Name, email, password, confirm password match, phone, gender, date of birth, address, emergency contact
Appointment Booking	Doctor selection, date selection, time slot selection
Edit Profile	Name, gender, date of birth, address, emergency contact format

4.8 Shared Logic

The custom hook src/hooks/useAppointmentForm.ts centralizes shared appointment logic across the booking and editing screens:

- Doctor list loading from the backend
- Patient ID retrieval from AsyncStorage
- Date handling and formatting
- Available time slot fetching based on selected doctor and date
- Appointment form validation before submission
- Navigation back to the appointment list after successful booking

5. System Integration

All three components — the backend, the web frontend, and the mobile application — are integrated through a shared RESTful API. The backend is the single source of truth for all data and business logic.

5.1 Integration Overview

Component	Communicates With	Protocol	Auth Mechanism
Angular Frontend	Node.js Backend	HTTPS REST API	JWT Bearer Token via HTTP Interceptor
React Native Mobile App	Node.js Backend	HTTPS REST API	JWT Bearer Token via Axios Interceptor
Backend	MongoDB	Mongoose ODM	Internal connection string

5.2 Authentication Flow Across Components

- A user or patient logs in through either the web frontend or mobile app
- Credentials are sent to the /auth/login or /auth/patientSignUp endpoint
- The backend validates credentials, generates a JWT access token and refresh token, and returns them
- The web frontend stores the token in localStorage; the mobile app stores it in Expo Secure Store
- All subsequent protected requests attach the token as an Authorization: Bearer header
- The backend auth middleware validates the token on each request
- The permission middleware checks whether the user's role permits the requested action

5.3 Role-Based Access Across Layers

Role	Web Frontend Access	Mobile App Access
Super Admin	Full access to all modules including employee creation, deletion, approvals, and all settings	Not applicable — admin roles use web interface
Admin	Employee management, patient management, appointment management, dashboard, approvals, medical records	Not applicable
Receptionist	Patient registration and management, appointment creation and updates, medical record access	Not applicable

Role	Web Frontend Access	Mobile App Access
Doctor	Appointment viewing, medical record creation and update for assigned patients	Not applicable
Patient	Not applicable — patients use mobile app	Login, doctor search, appointment booking and management, medical record viewing, profile management

5.4 Data Consistency

Because all clients (web and mobile) communicate with the same backend and MongoDB instance, data is consistent across all access channels in real time. A patient appointment created on the mobile app is visible to a receptionist on the web interface.

6. Functional Modules Summary

The following table summarizes the key functional modules and their implementation across all three platform layers:

Module	Backend Controller	Web Frontend Page	Mobile Screen
Authentication	auth.controller.js	login.ts, signup.ts	login.tsx, signup.tsx
Employee Management	admin.controller.js, user.controller.js	employee.ts, approval.ts	Not applicable
Patient Management	user.controller.js	patient.ts	profile.tsx, edit-profile.tsx
Appointment Management	appointment.controller.js	appointment.ts	appointment.tsx, view-appointment.tsx, edit-appointment.tsx
Medical Records	medical-record.controller.js	medical-record.ts	view-medical-record.tsx
Dashboard & Statistics	admin.controller.js	dashboard.ts	Not applicable
User Approval	admin.controller.js	approval.ts	Not applicable
Doctor Discovery	user.controller.js, appointment.controller.js	appointment.ts (doctor list)	home.tsx
Role & Navigation Metadata	ui.controller.js, node.controller.js	Dynamic sidebar	Not applicable

7. Error Handling and Validation Strategy

7.1 Backend Validation

Input validation is implemented using express-validator. Validation rules are attached to routes before the controller executes. Common validations include:

- Email format and uniqueness
- Required field presence
- Allowed role, department, and specialization values
- Date format and password strength requirements

7.2 Backend Error Handling

The backend uses a centralized error handling approach:

- Custom error objects are created through the centralized error factory in src/utils/errors.utils.js
- Controllers throw these custom errors instead of handling each case manually
- The asyncHandler wrapper ensures all unhandled promise rejections are forwarded to error middleware
- The error handler middleware formats all errors into a consistent JSON response with status code and message

7.3 Frontend Error Handling

The Angular frontend handles API errors by:

- Catching HTTP failures in Angular services using RxJS error operators
- Displaying user-friendly toast notifications using ngx-toastr
- Redirecting users to the access-denied page for permission failures
- Redirecting users to the 404 not-found page for invalid routes

7.4 Mobile Application Error Handling

The React Native mobile app handles errors through:

- The Axios interceptor displaying a user-friendly alert for all API error responses
- Local form validation providing inline feedback before any request is sent
- Toast messages confirming successful operations or communicating failures

7.5 Common Error Scenarios

Error Scenario	Backend Response	Client Handling
Invalid login credentials	401 Unauthorized	Toast notification: invalid email or password
Missing or expired JWT token	401 Unauthorized	Interceptor redirects user to login page
Insufficient permissions	403 Forbidden	Redirect to access-denied page
Invalid request body	422 Unprocessable Entity	Field-level validation messages displayed
Duplicate email or registration number	409 Conflict	Toast notification with error detail
Resource not found (patient, doctor, appointment)	404 Not Found	Toast notification or 404 page
Server or database error	500 Internal Server Error	Generic error toast message

8. AWS Deployment

The HMS platform is deployed on Amazon Web Services (AWS) using EC2 virtual machines. Continuous deployment is automated through GitHub Actions workflows that trigger on pushes to the sprint branch. The backend is served with PM2 and the frontend is served with Nginx.

10.1 Infrastructure Overview

Component	Hosting	Process Manager / Server	Branch Deployed
Backend (Node.js/Express)	AWS EC2 Instance	PM2	<branch-name>
Frontend (Angular)	AWS EC2 Instance	Nginx	<branch-name>
Database (MongoDB)	Configured separately	MongoDB service	N/A

10.2 CI/CD Pipeline

Both the backend and frontend use GitHub Actions for automated deployment. Each workflow is triggered on a push to the < branch-name > branch and connects to the EC2 instance over SSH using the appleboy/ssh-action.

10.2.1 Secrets Configuration

The following GitHub repository secrets are required for both workflows:

Secret Name	Purpose
EC2_HOST	Public hostname or IP address of the EC2 instance
EC2_USER	SSH username for the EC2 instance (e.g. ubuntu or ec2-user)
EC2_SSH_KEY	Private SSH key used to authenticate with the EC2 instance

10.2.2 Backend Deployment Workflow

File: .github/workflows/deploy-backend.yml

```
name: Deploy HMS Backend

on:
  push:
    branches:
      - <branch-name>
```

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy EC2
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ <secrets.EC2_HOST> }}
          username: ${{ <secrets.EC2_USER> }}
          key: ${{ <secrets.EC2_SSH_KEY> }}
          script: |
            cd hms-project/hms-backend-pod-3
            git pull origin <branch-name>
            npm ci
            pm2 restart hms-backend || pm2 start src/server.js --name
hms-backend
            pm2 save
```

10.2.3 Backend Deployment Steps Explained

Step	Command	Purpose
Pull latest code	git pull origin <branch-name>	Updates the backend source on the EC2 instance to the latest commit on the sprint branch
Install dependencies	npm ci	Installs exact dependency versions from package-lock.json for a clean, reproducible build
Restart application	pm2 restart hms-backend || pm2 start src/server.js --name hms-backend	Restarts the running PM2 process if it exists, or starts a new one if this is the first deployment
Save PM2 state	pm2 save	Persists the PM2 process list so the backend auto-starts after an EC2 reboot

10.2.4 Frontend Deployment Workflow

File: .github/workflows/deploy-frontend.yml

```
name: Deploy HMS FRONTEND

on:
  push:
    branches:
      - <branch-name>
```

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy EC2 FRONTEND
        uses: appleboy/ssh-action@v1
        with:
          host: ${{ <secrets.EC2_HOST> }}
          username: ${{ <secrets.EC2_USER> }}
          key: ${{ <secrets.EC2_SSH_KEY> }}
          script: |
            cd hms-project/hms-frontend-pod3
            git pull <branch-name>
            npm ci
            ng build --configuration production
            sudo rm -rf /var/www/hms/*
            sudo cp -r dist/hms-frontend/browser/* /var/www/hms/
            sudo systemctl restart nginx
```

10.2.5 Frontend Deployment Steps Explained

Step	Command	Purpose
Pull latest code	git pull origin <branch-name>	Updates the frontend source on the EC2 instance to the latest commit on the sprint branch
Install dependencies	npm ci	Installs exact dependency versions from package-lock.json for a clean, reproducible build
Build for production	ng build --configuration production	Compiles the Angular application with production optimizations and outputs to dist/hms-frontend/browser/
Clear old build	sudo rm -rf /var/www/hms/*	Removes the previous build artifacts from the Nginx web root to ensure a clean deployment
Copy new build	sudo cp -r dist/hms-frontend/browser/* /var/www/hms/	Copies the new compiled Angular application files into the Nginx web root
Restart Nginx	sudo systemctl restart nginx	Reloads Nginx to serve the updated frontend application

10.3 Deployment Trigger Summary

Workflow	Trigger Event	Target Branch	Deployment Target
Deploy HMS Backend	Push to branch	<branch-name>	EC2 — PM2 Node.js process
Deploy HMS Frontend	Push to branch	<branch-name>	EC2 — Nginx web server

9. Conclusion

The Hospital Management System has been implemented as a cohesive three-tier platform. The Node.js/Express backend provides a robust, layered API that enforces role-based authorization and maintains data integrity through structured Mongoose schemas. The Angular frontend delivers a responsive and secure management interface for all hospital staff roles. The React Native mobile application extends the platform to patients, enabling them to manage appointments and access their medical history from any mobile device.

The architecture is modular, maintainable, and aligned with the requirements defined in the HMS Business Requirements Document. Role-based access control is enforced at every layer — from route guards in the frontend to permission middleware on the backend — ensuring that all operations are performed only by authorized users. The shared API design between the web and mobile clients ensures data consistency across all access channels.